SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

SSAFY

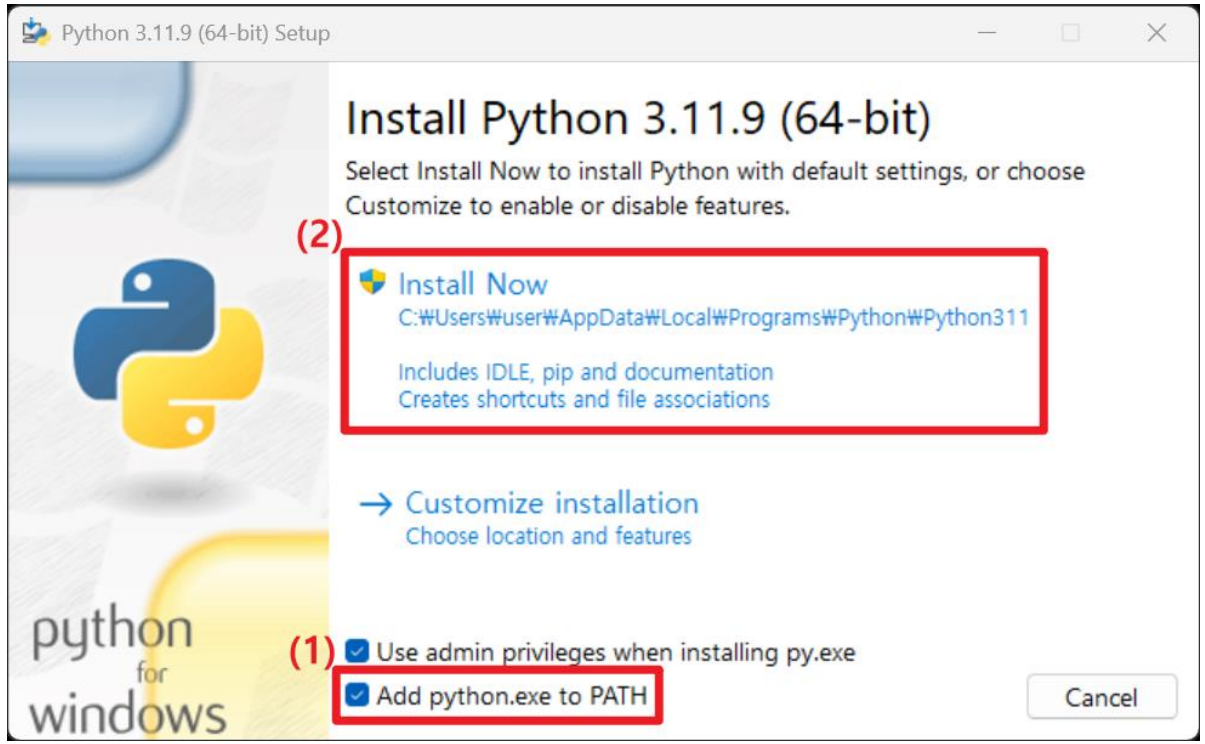
AI 코딩 어시스턴트
사전 준비 및 학습 자료

AI 강의 I

사전 준비

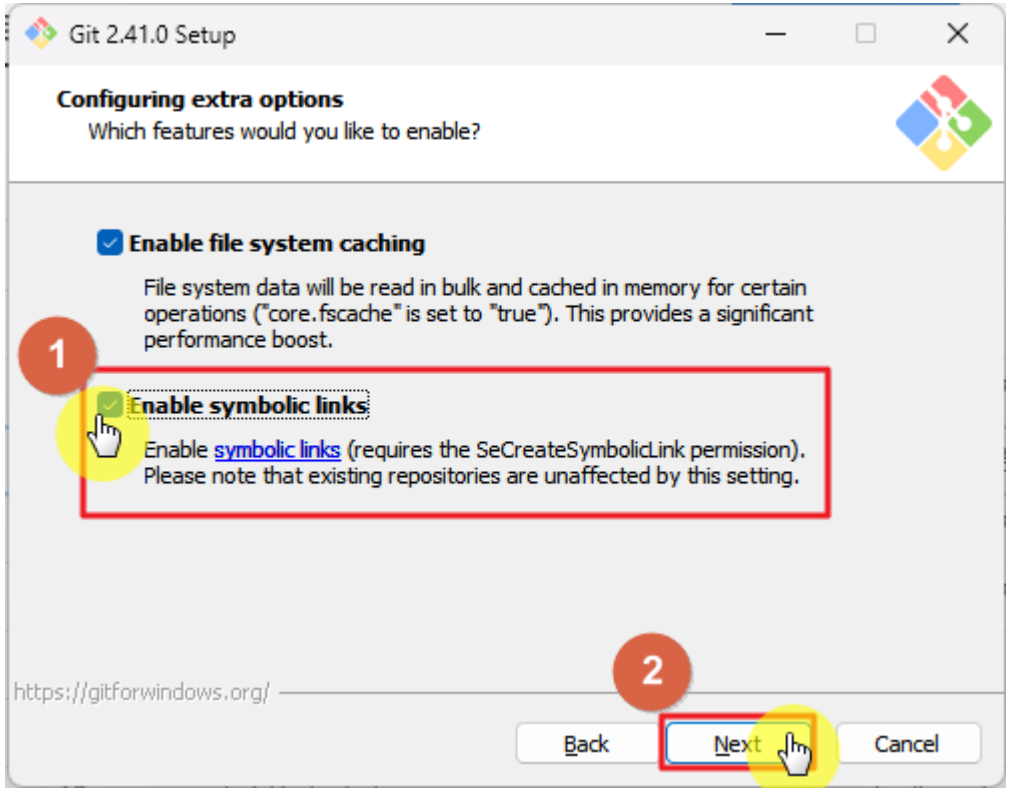
python 3.11 이상 설치

- 바탕화면 > 설치파일 > python 3.11 설치
- Add python.exe to PATH에 먼저 체크하고, Install Now를 클릭해 python을 설치

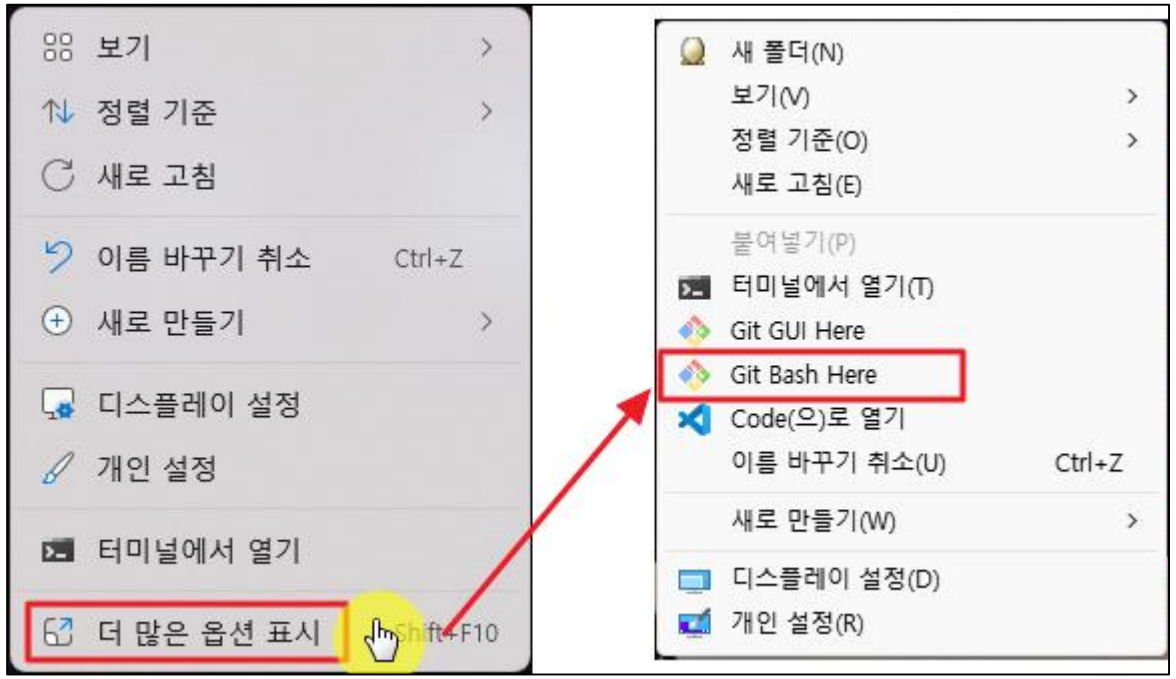


Git Bash 설치

- 바탕화면 > 설치파일 > Git Bash 설치

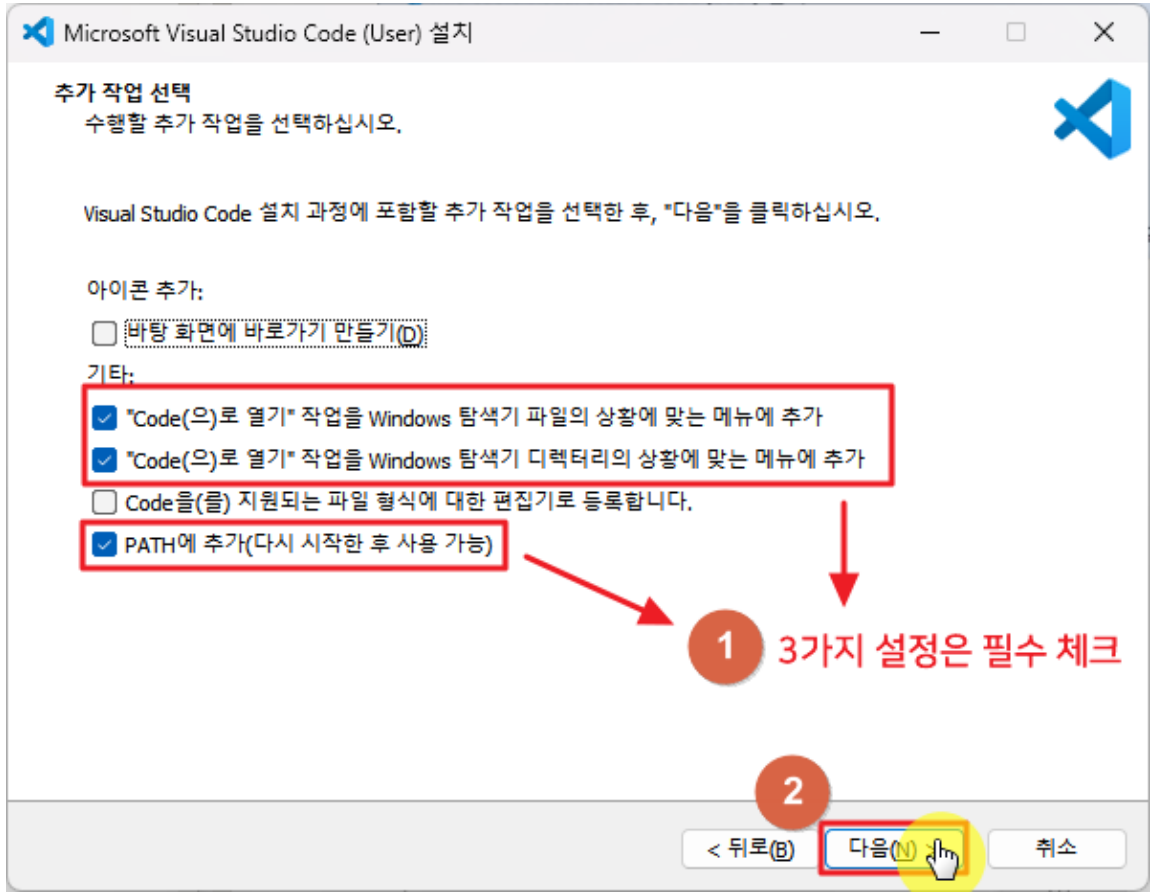


※ 설치 완료 후, Code(으)로 열기 옵션을 클릭했을 때 VSCode가 정상적으로 실행되는지 확인

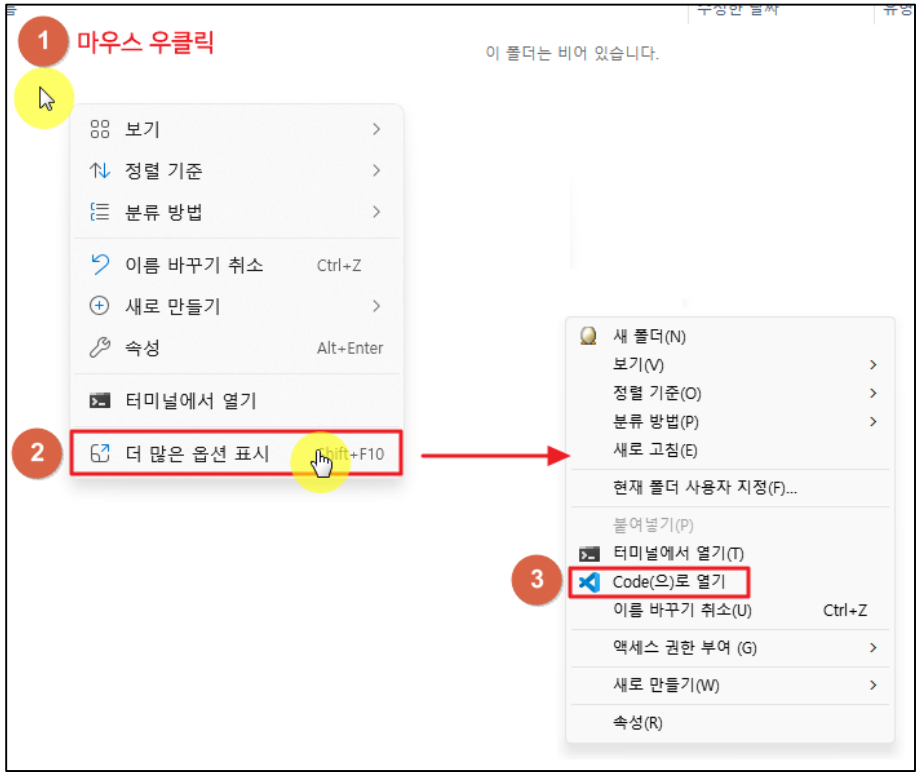


VS Code 설치

- 바탕화면 > 설치파일 > VS Code 설치

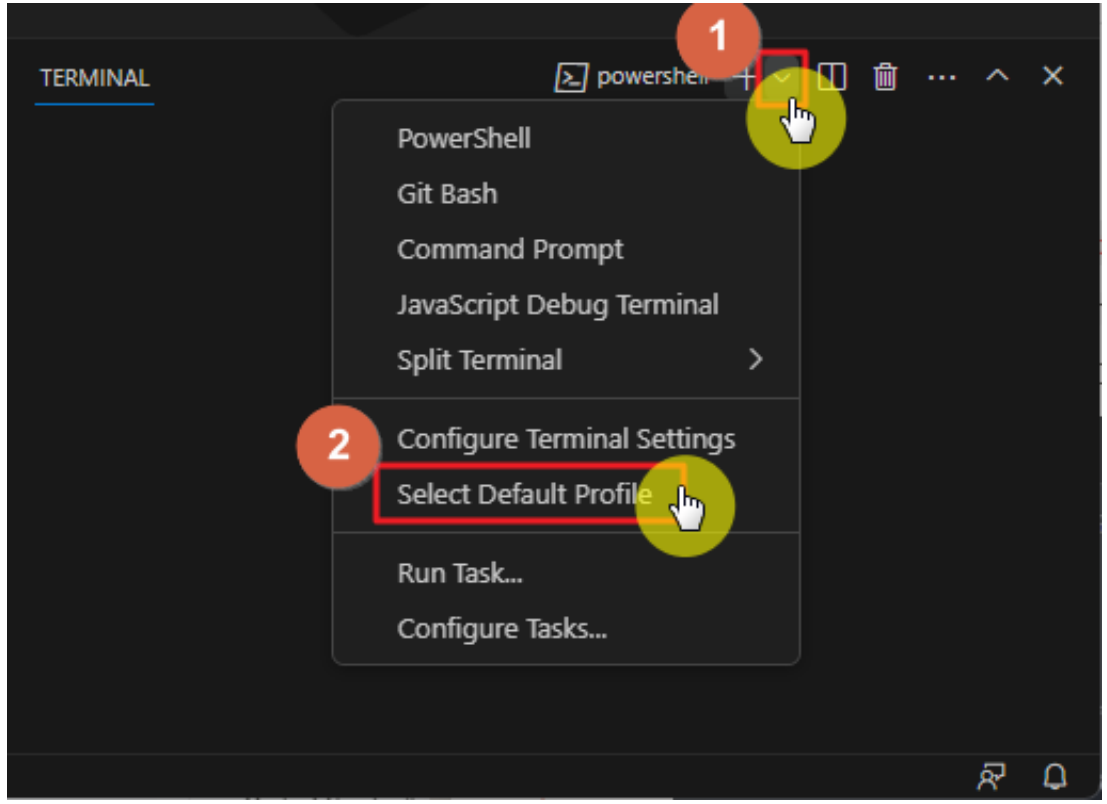


※ 설치 완료 후, Code(으)로 열기 옵션을 클릭했을 때 VS Code가 정상적으로 실행되는지 확인

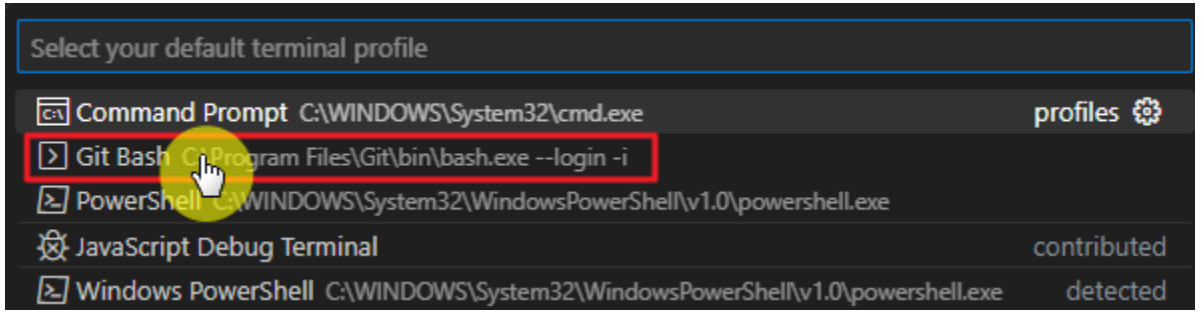


VS Code 설정 - 기본 터미널 지정

- VS Code 실행 후, 'ctrl + `(백틱)' 으로 터미널 실행 - 기본 터미널을 git bash로 수정



※ 설정 후, 터미널을 완전히 종료 (□) 후, 실행



SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

Day 02

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로! Enabling People

LLM에서 작성 해보기

LLM에서작성 해보기 (1/8)

"Node.js로 사용자 CRUD API 만들어줘"

LLM에서작성 해보기 (2/8)

Node.js + Express 기반으로
사용자 CRUD API를 구현하라.
입력 검증 포함.
RESTful 규칙 준수.
Jest 기반 테스트 코드 작성.
코드 구조는 controller/service 분리.

LLM에서작성 해보기 (6/8)

"비동기 로직을 추가하고, 에러 핸들링을 강화해줘"

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로! Enabling People

LLM 실습 1
- 역할별 프롬프트 생성하기

LLM 실습 1 - 역할별 프롬프트 생성하기 (7/8)

```
너의 작업은 역할별 프롬프트 3개(PLANNER/GENERATOR/REVIEWER)를 '완성본'으로 작성하는 것이다.

[프로젝트 맥락]
HTML/CSS/JS를 활용한 커뮤니티 웹 서비스
[공통 규칙]
- 프롬프트는 복사해서 바로 사용할 수 있어야 한다.
- 각 프롬프트는 역할 / 목표와 금지사항 / 입력 맥락 / 출력 구조 /  예시 섹션을 반드시 포함한다.

[역할별 핵심 규칙]
- PLANNER: 요구사항 정리 및 요약 / 제약 정의만 수행 /  MUST NOT 코드 작성.
- GENERATOR: 계획을 기반으로 산출물 생성 / 단계별 기능 구현 / MUST NOT 계획 수정, 불필요한 파일 수정, 추측 구현
- REVIEWER: 산출물 평가 / 리스크 / 개선점만 제시 / MUST NOT 코드 수정본 작성, 새로운 계획 제안
[각 프롬프트에 반드시 포함할 내용]
1) 역할
  - 너는 {ROLE_NAME}이다.
  - 목표 산출물: {OUTPUTS}
2) 목표와 금지사항
  - MUST / MUST NOT를 명시
3) 입력 맥락
   - 입력으로 받을 것: {INPUTS}
4) 출력 맥락
  - 응답 템플릿을 고정(예: 목록/체크리스트/diff 등)
5) 예시 (최소 1개)

[FEW-SHOT: META]
User: HTML/CSS/JS를 활용한 커뮤니티 웹 서비스 역할별 프롬프트(PLANNER/GENERATOR/REVIEWER) 만들어줘.
Assistant: (PLANNER/GENERATOR/REVIEWER 각각에 대해 위 섹션을 포함한 프롬프트 3개를 코드블록으로 출력)

[출력 형식]
- 아래 순서로 마크다운 블럭 3개만 출력하라.
 1) PLANNER Prompt
 2) GENERATOR Prompt
 3) REVIEWER Prompt
- 각 마크다운 블럭 첫 줄에는 "이 프롬프트의 사용 목적"을 한 줄로 적어라.
```

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

Day 03

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로! Enabling People

일단 사용해보기 1
- 기능추가

일단 사용해보기 1 - 기능추가 (2/3)

현재 프로젝트에는 로그아웃 기능이 없다. 기능을 추가한다.

일단 사용해보기 2 - 레이아웃 변경 (1/2)

전체 레이아웃과 스타일을 변경한다.
각 화면이 전체 화면 크기에 맞아야 하고,
전반적인 스타일이 파스텔 톤으로 변경되어야 한다.
최신 트렌드를 반영한 스타일로 변경

사용자 지정 지침 작성하기

사용자 지정 지침 작성하기 전체 프롬프트

```
# Copilot 시스템 라우팅 지침

## 기본 원칙
당신은 이 프로젝트의 AI 파트너입니다. 사용자의 요청에 직접 답변하기 전에, 반드시 **의도를 분류**하고 지정된 **규칙 파일(Rule File)을 우선적으로 참고**해야 합니다.

## 라우팅 및 파일 참조 규칙
사용자의 요청을 분석하여 아래 3가지 모드 중 하나를 선택하고, 해당 경로의 파일 내용을 바탕으로 답변하세요.

### 1. 기획 모드 (PLANNER)
- **트리거:** 요구사항 정의, 기능 설계, 기술 검토, "어떻게 만들까?"
- **참조 파일:** `.github/rules/planner.md`
- **지침:** 위 파일을 찾아 그 안에 정의된 계획 수립 절차를 따르세요. 코드를 작성하지 말고 설계안을 제시하세요.

### 2. 생성 모드 (GENERATOR)
- **트리거:** 코드 작성, 기능 추가, 버그 수정, "이거 만들어줘"
- **참조 파일:** `.github/rules/generator.md`
- **지침:** 위 파일을 찾아 그 안에 정의된 코딩 컨벤션과 스타일 가이드를 엄격히 준수하여 코드를 작성하세요.

### 3. 검증 모드 (REVIEWER)
- **트리거:** 코드 리뷰, 테스트 케이스 생성, 보안 점검, "이 코드 괜찮아?"
- **참조 파일:** `.github/rules/reviewer.md`
- **지침:** 위 파일을 찾아 그 안에 정의된 검증 체크리스트를 기반으로 코드를 평가하세요.

## 예외 처리
- 사용자의 요청이 모호하거나 위 3가지 분류에 속하지 않으면, 일반적인 도움말을 제공하되 "어떤 모드로 도와드릴까요?"라고 되물어 확인하세요.
```

사용자 지정 지침 작성하기 (5/5)

현재 적용된 지침이 무엇인지 알려줘

역할별 지침 작성하기

Planner 지침 작성하기

```
# PLANNER 가이드

## 할 일 (Do)
- 사용자의 목표를 **한 문장**으로 명확히 재정의하세요.
- 불명확한 점에 대한 가정이나 질문을 나열하세요 (**최대 3개**).
- 작업 범위를 실행 가능한 작은 단계로 제안하세요 (**최대 3개**).
- 작업 완료를 판단할 수 있는 인수 조건(Acceptance Criteria)을 정의하세요 (**최대 3개**).

## 하지 말아야 할 일 (Do Not)
- **코드를 변경하거나 생성하지 마세요.**
- 요청된 범위와 관련 없는 리팩토링이나 개선 제안을 하지 마세요.

## 출력 형식
- **목표 (Goal):**
- **질문 (Questions):**
- **범위 (Scope - Steps):**
- **인수 조건 (Acceptance Criteria):**

---
**마지막 작업:**
작성된 기획 내용을 `docs/plan.md` 파일에 저장하거나, 이미 파일이 존재할 경우 **기존 내용 하단에 추가(Append)**할 수 있는 쉘 명령어(Shell Command)를 제공하세요.
```

Generator 지침 작성하기 (.github/rules/generator.md)

```
# GENERATOR 가이드

## 할 일 (Do)
- 기획(Scope) 단계에서 정의된 변경 사항만 정확히 구현하세요.
- **최소한의 변경(Minimal Diff)**과 최소한의 파일 수정을 지향하세요.
- 기존 프로젝트의 코딩 스타일과 디렉토리 구조를 철저히 유지하세요.

## 하지 말아야 할 일 (Do Not)
- **파일 전체를 다시 작성(Rewrite)하지 마세요.** (변경된 부분만 명시)
- 명시적인 요청 없이 코드를 '정리(Cleanup)'하거나 리팩토링하지 마세요.
- 허락 없이 API나 DB 스키마에 하위 호환성을 깨는 변경(Breaking Change)을 하지 마세요.

## 출력 형식
- **변경 파일 목록 (Changed Files):**
- **변경 요약 (Diff Summary - Why & What):**
- **검증 방법 (How to Verify - Tests/Commands):**
- **잠재적 위험 (Risks):**
```

Reviewer 지침 작성하기 (.github/rules/reviewer.md)

```
# REVIEWER 가이드

## 할 일 (Do)
- **코드를 절대 직접 수정하지 마세요.** (눈으로만 확인)
- 변경 사항(Diff)의 영향 범위를 검토하세요:
  - **동작 방식 (Behavior):** 의도한 대로 작동하는가?
  - **인터페이스 (Interface):** 입출력 값이 올바른가?
  - **위험 요소 (Risk):** 보안 취약점이나 성능 저하 가능성은 없는가?
  - **롤백 (Rollback):** 문제 발생 시 되돌릴 수 있는가?
- 누락된 테스트 케이스와 엣지 케이스(예외 상황)를 지적하세요.

## 하지 말아야 할 일 (Do Not)
- **직접 구현 코드를 작성하지 마세요.**
- 현재 작업 범위를 벗어난 대규모 재설계(Redesign)를 제안하지 마세요.

## 출력 형식
- **변경 요약 (Summary of change):**
- **위험 요소 및 엣지 케이스 (Risks/Edge cases):**
- **테스트 공백 (Test gaps - 테스트가 필요한 부분):**
- **수정 제안 (Suggested fixes - 텍스트 설명만):**

---
**마지막 작업:**
작성된 리뷰 내용을 `docs/review.md` 파일에 저장하거나, 이미 파일이 존재할 경우 **기존 내용 하단에 추가(Append)**할 수 있는 쉘 명령어(Shell Command)를 제공하세요.
```

Planner 테스트

게시판을 좀 더 쓸만하게 개선하려면 뭐부터 손대면 돼?

검색 기능 넣고 싶은데, 어디를 어떻게 해야 할까?

스타일링이 너무 마음에 안 들어. 어떻게 수정하면 좋을까?

Planner 테스트 결과 확인과 추가 요청

기존의 로직을 최대한 유지하면서,
게시글을 검색할 수 있는 기능이 추가 될 수 있도록 계획서를 파일에 작성해 줘

Generator 테스트

방금 작성한 plan.md 파일을 참고하여, 코드를 수정 해줘.

Reviewer 테스트

이 프로젝트에서 가장 취약한 부분이 무엇인지 확인하고,
Planner가 참고해서 다음 계획을 작성할 수 있도록 md 파일로 정리 해 줘

SAMSUNG SW AI ACADEMY FOR YOUTH

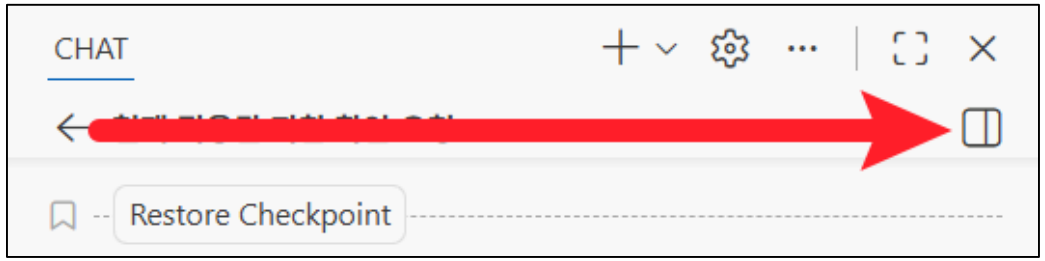
함께가요 미래로! Enabling People

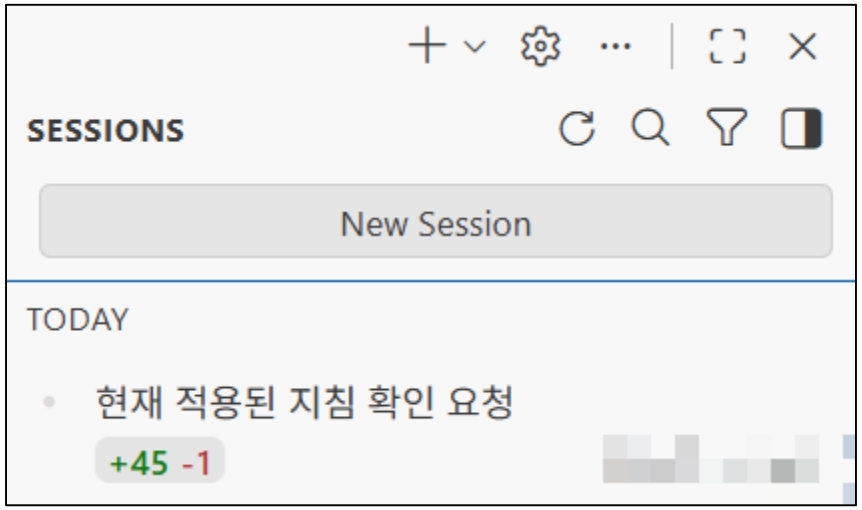
세션 나눠서 작업하기

함께가요 미래로!
Enabling People

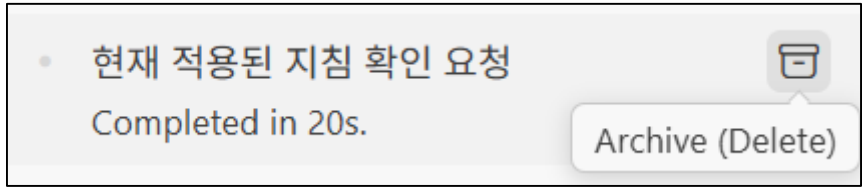
세션 나눠서 작업하기

- VS Code Copliot Chat은 한 프로젝트에서 여러 세션을 사용할 수 있음





- 기존에 사용 중이던 세션 목록 확인
- 세션 목록을 통해 세션을 변경하며 작업가능
- 기존 세션들을 archive (delete) 할 수 있음



기존 세션을 삭제하고, 역할별 세션 생성하기

- New Session으로 새로운 세션을 활성화
- 각 세션마다 .github/rules/{planner, generator, reviewer}.md를 참고하도록 지시

> 너는 PLANNER 역할만 수행한다. .github/rules/planner.md 규칙만 따른다.
> 코드 작성/수정 금지. 출력은 계획(목표/범위/성공기준/질문)만 작성.

> 너는 GENERATOR 역할만 수행한다. .github/rules/generator.md 규칙만 따른다.
> 최소 변경으로만 구현. 리팩토링/전체 재작성 금지. 변경 요약과 검증 방법 포함.

> 너는 REVIEWER 역할만 수행한다. .github/rules/reviewer.md 규칙만 따른다.
> 코드 수정 금지. diff 기준 영향/리스크/누락 테스트만 평가. 수정 제안은 텍스트로만 작성.

참고 - git diff 기반 판단

참고 - git diff 기반 판단 사전준비

- 'ctrl + shift + `(백틱)' 으로 새로운 터미널을 열고, 아래 명령어를 차례로 작성

```
git init
```

```
git add .
```

```
git commit -m "first commit"
```

코드 변동 사항 만들기

- generator 세션으로 프로젝트 스타일 변경하기

```
프로젝트의 전체 테마를 흑백으로 바꿔줘.
```

git diff 확인하기

- 코드 변경 사항을 저장 (keep)하고 터미널에 git diff 명령어 입력

```
git diff
```

git diff 기반으로 작업 지시하기

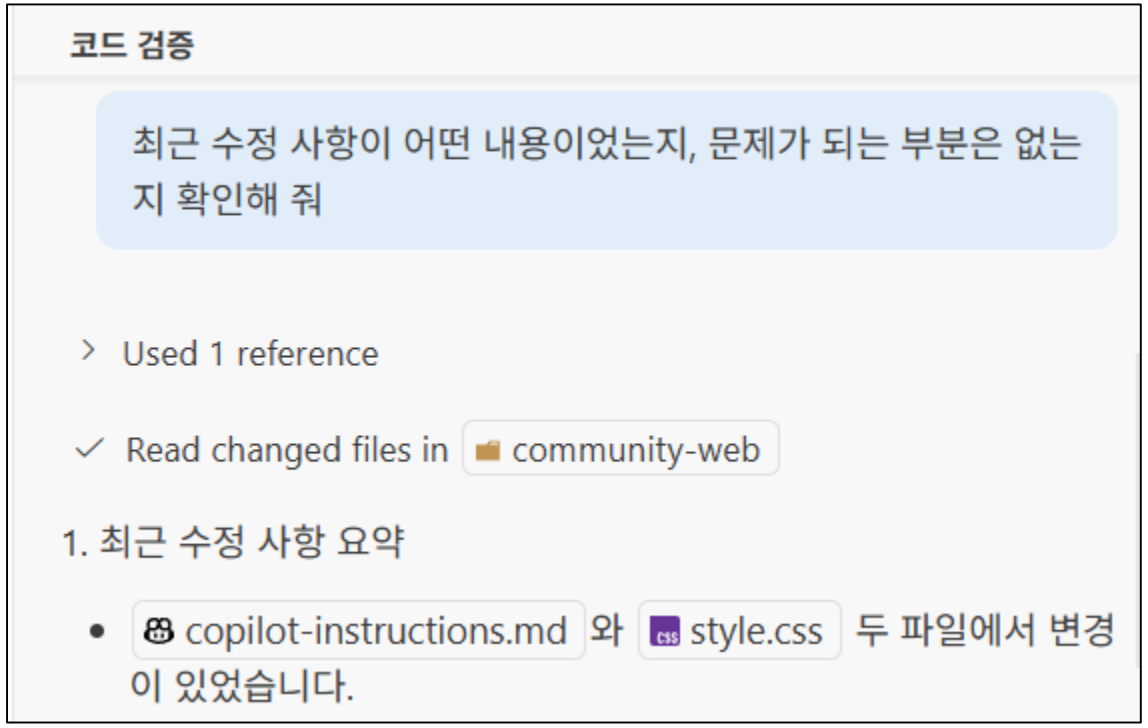
- 공통 지침 (.github/copilot-instructions.md)을 수정

```
# Copilot 시스템 라우팅 지침

## 최우선 제약 사항 (Must Follow)
모든 답변을 생성하기 전에, 반드시 **현재의 Git 변경 사항(git diff)**을 먼저 분석하세요.
1. 작업 트리에 커밋되지 않은 변경 사항이 있다면, 그것이 현재 작업의 **절대적인 기준점**입니다.
2. 사용자의 요청이 모호할 경우, 새로운 코드를 작성하기보다 **현재 변경된 코드(Diff)를 기반으로** 추론하세요.

## 기본 원칙
당신은 이 프로젝트의 AI 파트너입니다. 사용자의 요청에 직접 답변하기 전에 의도를 분류하고, 지정된 **규칙 파일(Rule File)**을 참고하여 답변해야 합니다.

## 라우팅 및 파일 참조 규칙
사용자의 요청을 분석하여 아래 3가지 모드 중 하나를 선택하세요.

### 1. 기획 모드 (PLANNER)
- **트리거:** 요구사항 정의, 기능 설계, "어떻게 만들까?", "계획 수정해줘"
- **참조 파일:** `.github/rules/planner.md`
- **지침:** `git diff`로 파악된 변경 사항을 기존 계획(`docs/plan.md`)에 반영하거나 동기화하세요.

### 2. 생성 모드 (GENERATOR)
- **트리거:** 코드 작성, 기능 추가, "이거 만들어줘", "수정해줘"
- **참조 파일:** `.github/rules/generator.md`
- **지침:** `git diff` 상태를 파악하여, 충돌 없이 최소한의 변경(Minimal Diff)만 발생하도록 코드를 작성하세요.

### 3. 검증 모드 (REVIEWER)
- **트리거:** 코드 리뷰, "이 코드 괜찮아?", "변경 사항 확인해줘"
- **참조 파일:** `.github/rules/reviewer.md`
- **지침:** 파일 전체가 아닌, `git diff`로 확인되는 **변경된 라인**을 중점적으로 검증하세요.

## 예외 처리
- 사용자의 요청이 모호하거나 위 3가지 분류에 속하지 않으면, "현재 변경 사항(Diff)을 기준으로 작업을 진행할까요?"라고 되물어 확인하세요.
```

지침 변경 사항 확인

- 완전히 다른 세션인 reviewer가 수정 사항을 잘 이해하는지 확인

최근 수정 사항이 어떤 내용이었는지,
문제가 되는 부분은 없는지 확인해 줘



SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

day 04

사전 준비

uv 설치

- PowerShell을 "관리자 권한"으로 열고 아래 명령어 실행

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

- 설치 확인

```
uv --version
```

TIP

- uv는 Python 프로젝트를 빠르게 관리하기 위한 패키지 및 가상환경 관리 도구입니다.
- pip, venv, virtualenv 등의 기능을 하나로 통합해 의존성 설치와 환경 생성을 단순화합니다.
- Rust로 작성되어 속도가 빠르고, 재현 가능한 개발 환경 구성을 목표로 합니다.

uv로 가상환경 생성과 활성화

- VS Code 터미널에서 uv로 가상환경 생성 (.venv는 가상환경 경로)

```
uv venv .venv
```

```
ssafy@DESKTOP-EJC7M8M MINGW64
$ uv venv .venv
Using CPython 3.12.12
Creating virtual environment at: .venv
Activate with: .venv\Scripts\activate
```

- 가상환경 활성화

```
source .venv/Scripts/activate
```

```
ssafy@DESKTOP-EJC7M8M MINGW64
$ source .venv/Scripts/activate
```

```
(.venv)
ssafy@DESKTOP-EJC7M8M MINGW64
$
```

TIP

- 가상환경은 프로젝트별로 Python과 라이브러리(의존성)를 격리해 충돌을 막는 폴더를 의미합니다.
- 같은 PC에서도 프로젝트마다 다른 버전 조합을 안전하게 쓸 수 있게 합니다.
- 삭제/재생성이 쉬워 "깨끗한 상태로 다시 시작"하기 좋습니다.

uv로 Langflow 설치

- 가상환경에서 Langflow 설치 (.venv 활성화 확인 필수)
  - 설치 환경에 따라 다소 시간이 소요될 수 있음

```
uv pip install langflow -U
```

- 설치 버전 확인 - (수업일 기준 1.8 이하 버전이 설치된 경우, 다음 장 명령어 추가 실행)
  - 최초 실행 시 다소 시간이 소요될 수 있음
  - 버전이 표기될 때까지 대기

```
uv run langflow --version
```

```
ssafy@DESKTOP-EJC7M8M MINGW64
$ uv run langflow --version
D:\download\ai\.venv\Lib\site-
y. Please upgrade your Pydanti
nation v0.16.0.
  from fastapi_pagination impo
langflow 1.7.3
```

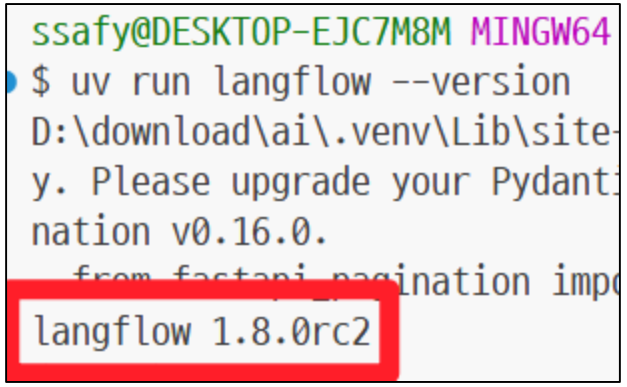
[추가] Langflow 버전 업데이트

- pre-release(프리릴리즈)로 업데이트
  - 설치 환경에 따라 다소 시간이 소요될 수 있음

```
uv pip install langflow --pre -U
```

- 설치 버전 확인
  - 최초 실행 시 다소 시간이 소요될 수 있음
  - 버전이 표기될 때까지 대기
  - 설치된 세부 버전은 스크린샷과 다를 수 있음

```
uv run langflow --version
```



TIP

- rc는 Release Candidate의 약자입니다.
- 정식 릴리스 직전 단계로, 큰 변경은 멈추고 최종 검증을 위해 배포되는 후보 버전입니다.

왜 pre-release를 사용하나요?

- 정식 버전에 아직 반영되지 않은 최신 기능이나 개선 사항을 먼저 사용할 수 있음
  - 1.7.3 버전 기준 PlayGround가 정상 실행되지 않는 이슈가 있음
- 버그 수정이나 성능 개선이 빠르게 포함되기 때문에 문제 해결이 필요한 경우 유리함
- 공식 릴리스 전에 새로운 변경 사항을 미리 검증해볼 수 있음
  - 다만 안정성이 완전히 보장되지는 않으므로 실험, 테스트 목적에 더 적합함

Langflow 실행

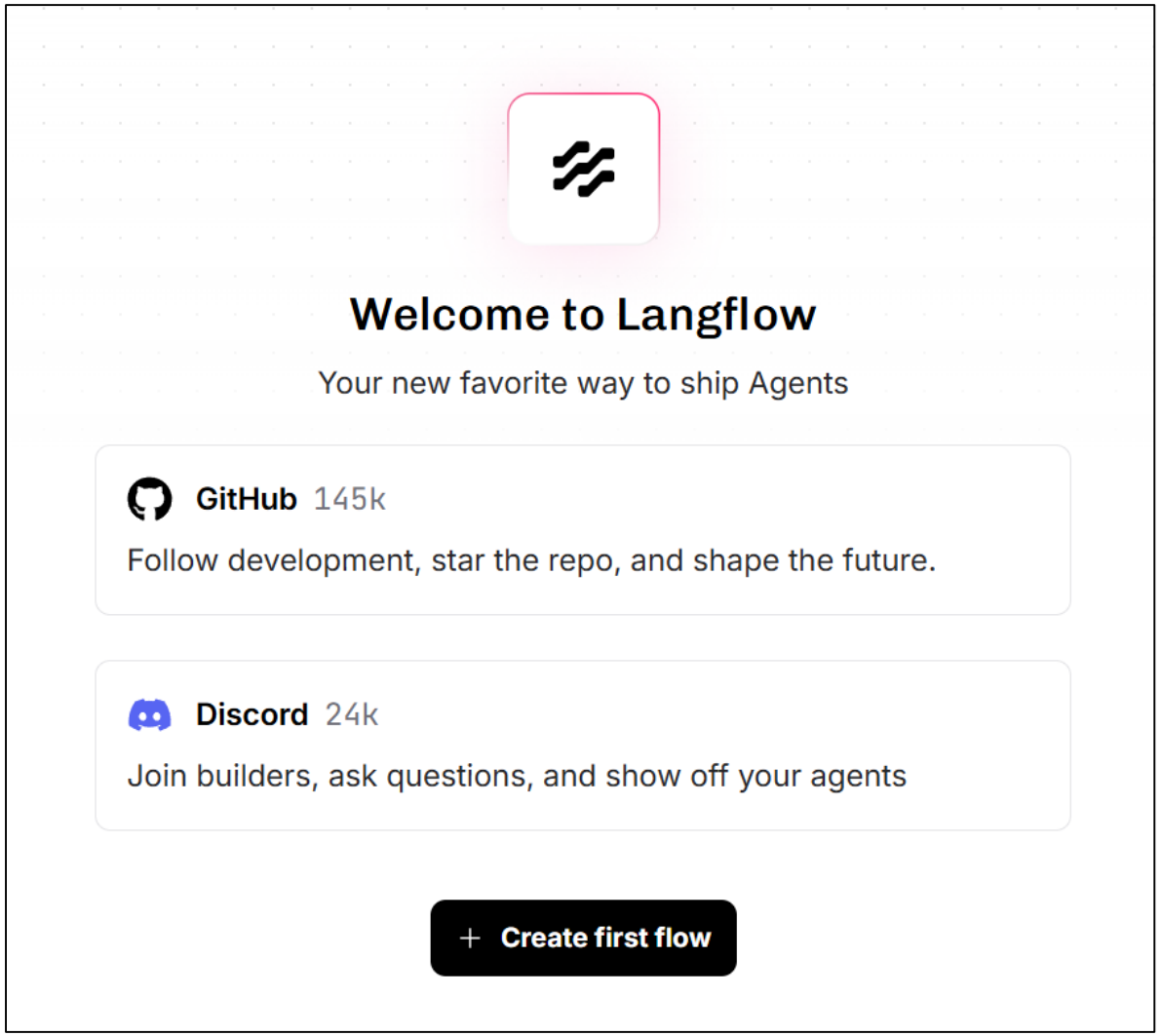
- 설치 완료 후, 아래 명령어로 Langflow 실행
  - 실행 완료 후, 초기화에 시간이 소요됨
  - http://localhost:7860 경로로 접속

```
uv run langflow run
```

```
Welcome to Langflow

* GitHub: Star for updates -> https://github.com/langflow-ai/langflow
# Discord: Join for support -> https://discord.com/invite/EqksyE2EX9

We collect anonymous usage data to improve Langflow.
To opt out, set: DO_NOT_TRACK=true in your environment.

[OK] Open Langflow -> http://localhost:7860
```

Langflow 실행

- http://localhost:7860 경로로 접속 시
- 우측 이미지와 같은 화면이 아니라면
  - 연결 할 수 없다는 화면이라면
- 잠시 대기 후, 새로고침



SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로! Enabling People

LLM 서비스 설계 실습 1
- 프롬프트 변경

LLM 서비스 설계 실습 1 - 프롬프트 변경

- Prompt Template 내부의 Template 변경하기

말 끝에 항상 '냥'을 붙여서 답변해 답변은 최대 3줄 까지만 가능

- PlayGround에서 사용자 입력 전송

안녕 내 이름은 홍길동이야

- 대화를 기억하는지 확인하기

내 이름이 뭐라고 했지?

SAMSUNG SW AI ACADEMY FOR YOUTH

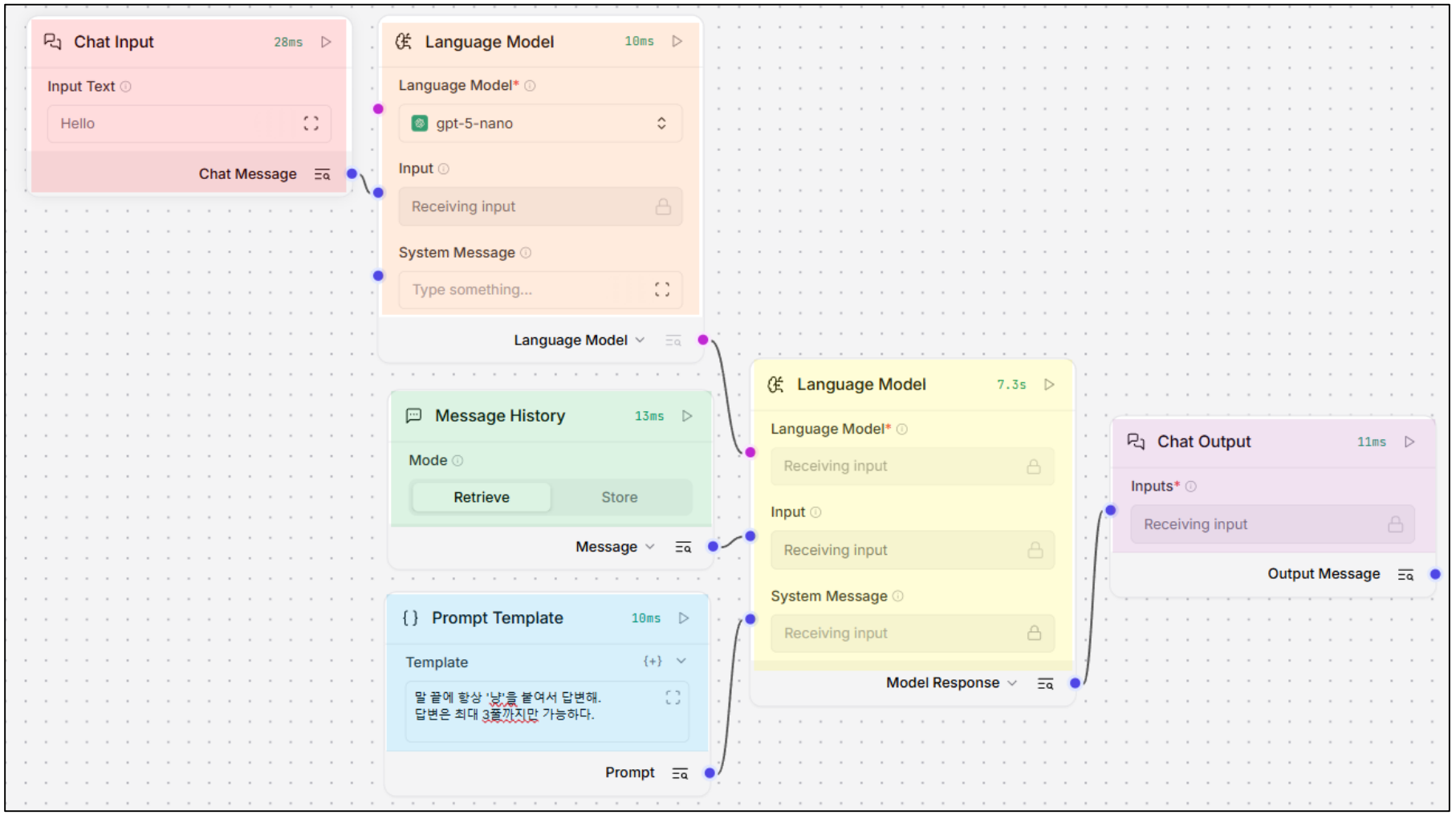
함께가요 미래로! Enabling People

LLM 서비스 설계 실습 2 - 기억 주입

LLM 서비스 설계 실습 2 - 기억 주입

- 전체 연결 설계도

1. 사용자 입력
2. 모델 → 모델
3. History 확인
4. 프롬프트 적용
5. 출력



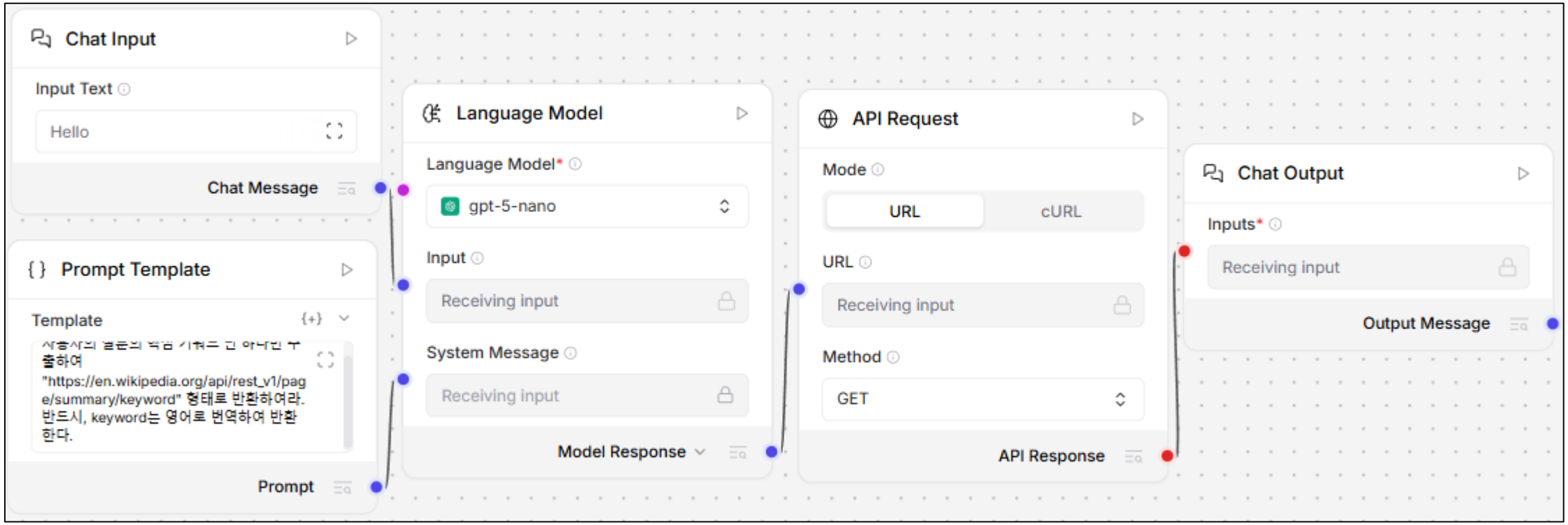
SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로! Enabling People

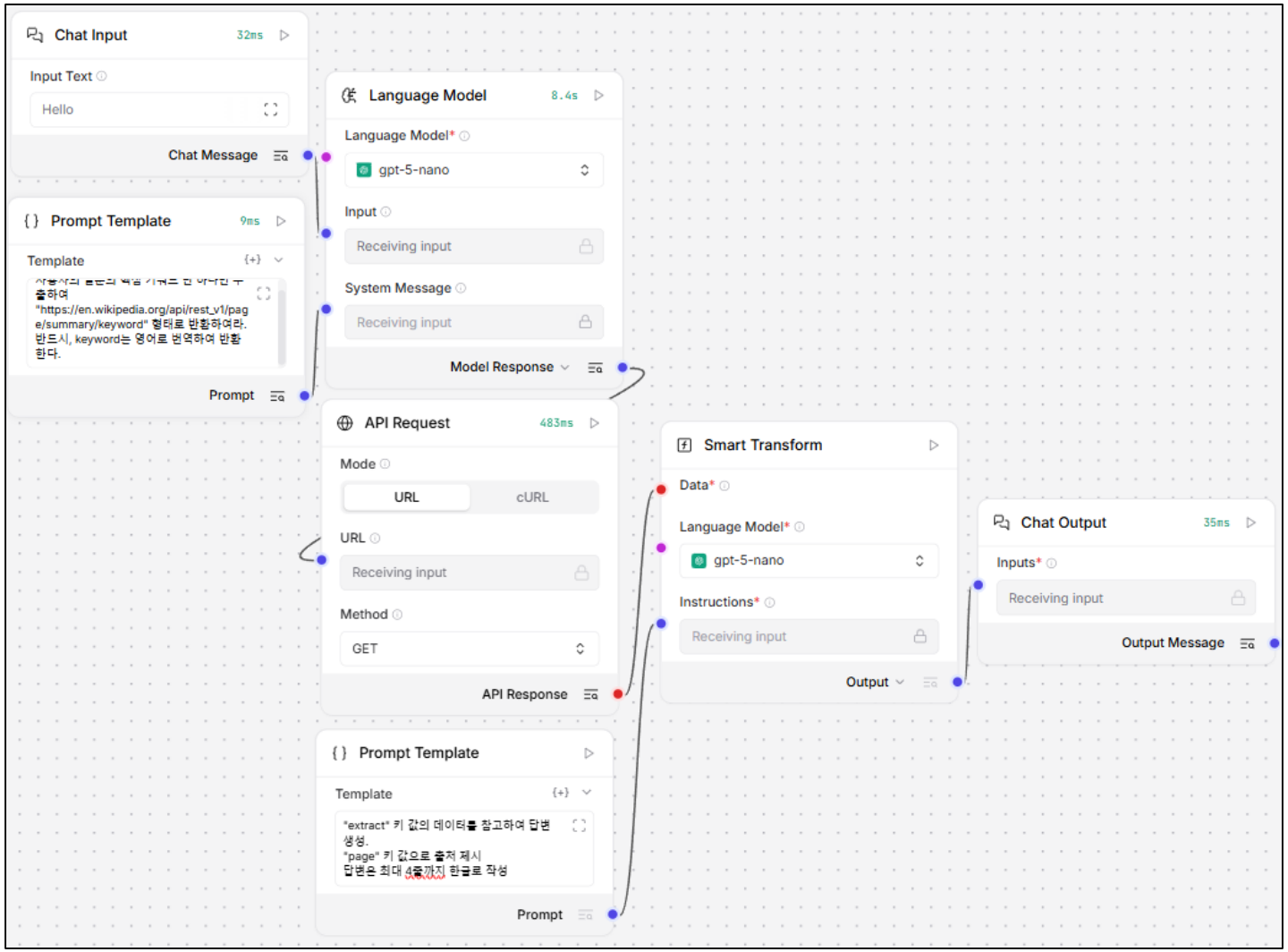
LLM 서비스 설계 실습 3 - 도구 추가

LLM 서비스 설계 실습 3 - 도구 추가 (2/8)

사용자의 질문의 핵심 키워드 단 하나만 추출하여
"https://en.wikipedia.org/api/rest_v1/page/summary/keyword" 형태로 반환하여라
반드시, keyword는 영어로 번역하여 반환한다.



LLM 서비스 설계 실습 3 - 도구 추가 (7/8)



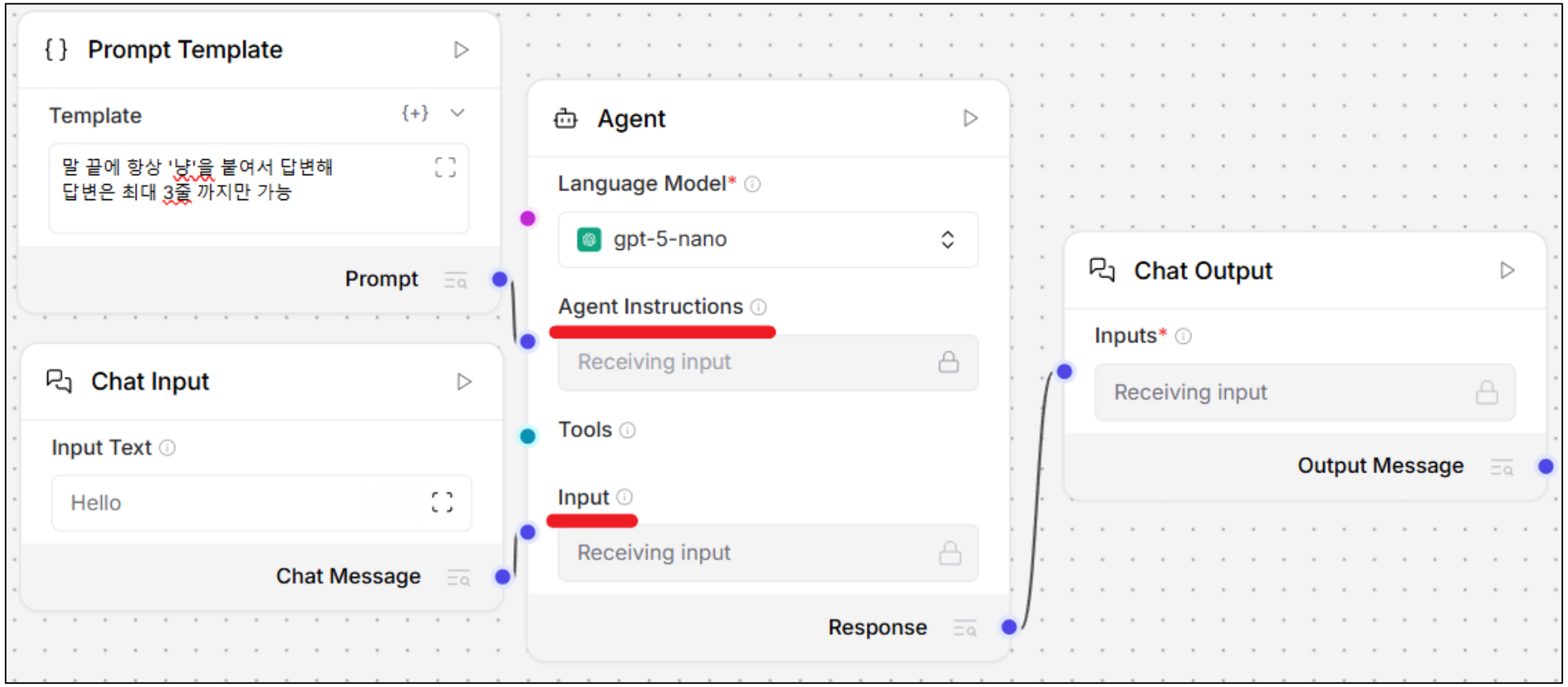
SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로! Enabling People

Agent 서비스 설계 실습 1

Agent 서비스 설계 실습 1

말 끝에 항상 '냥'을 붙여서 답변해
답변은 최대 3줄 까지만 가능



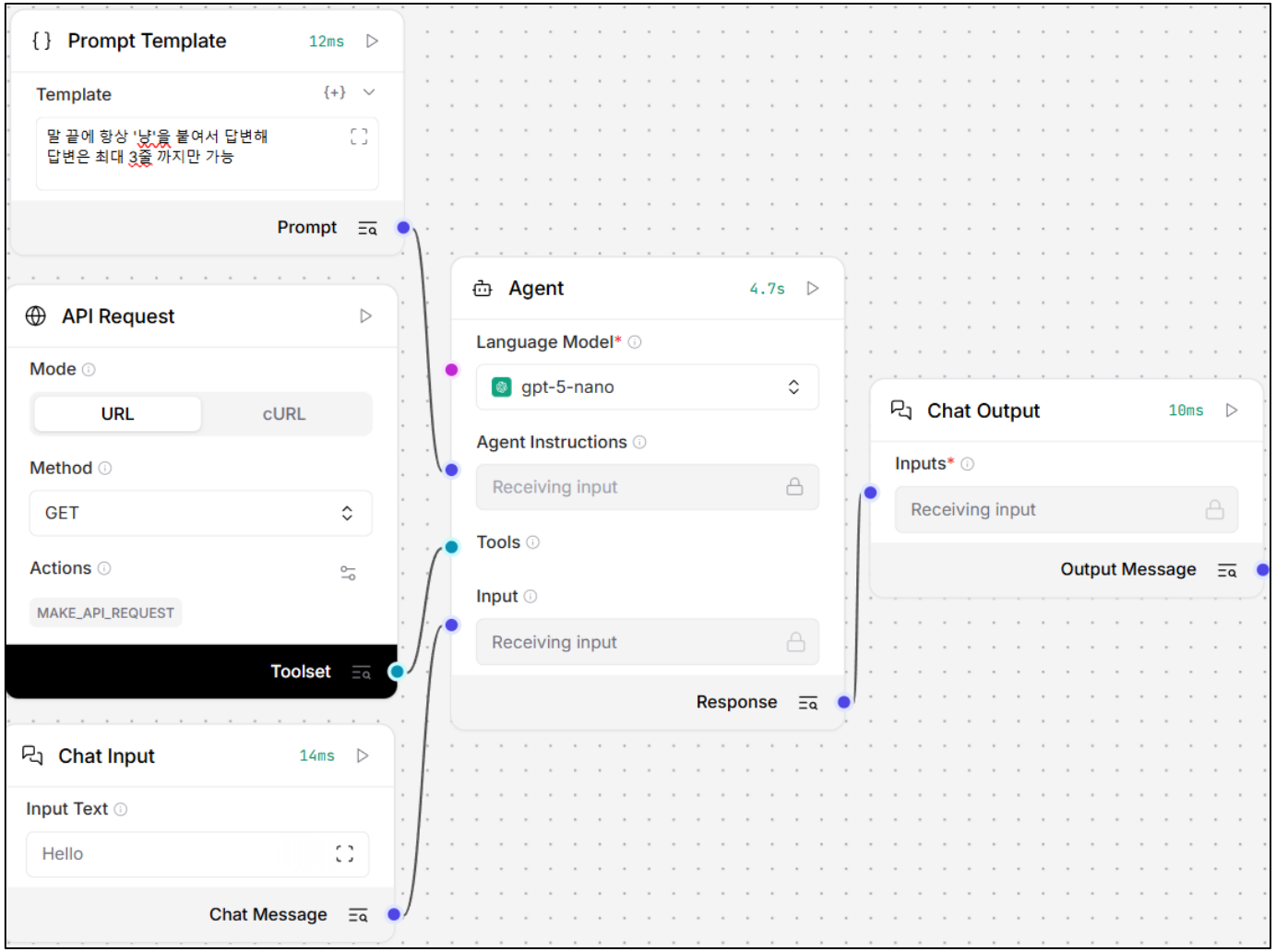
SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로! Enabling People

Agent 서비스 설계 실습 2 - 도구 추가

Agent 서비스 설계 실습 2 - 도구 추가(2/3)

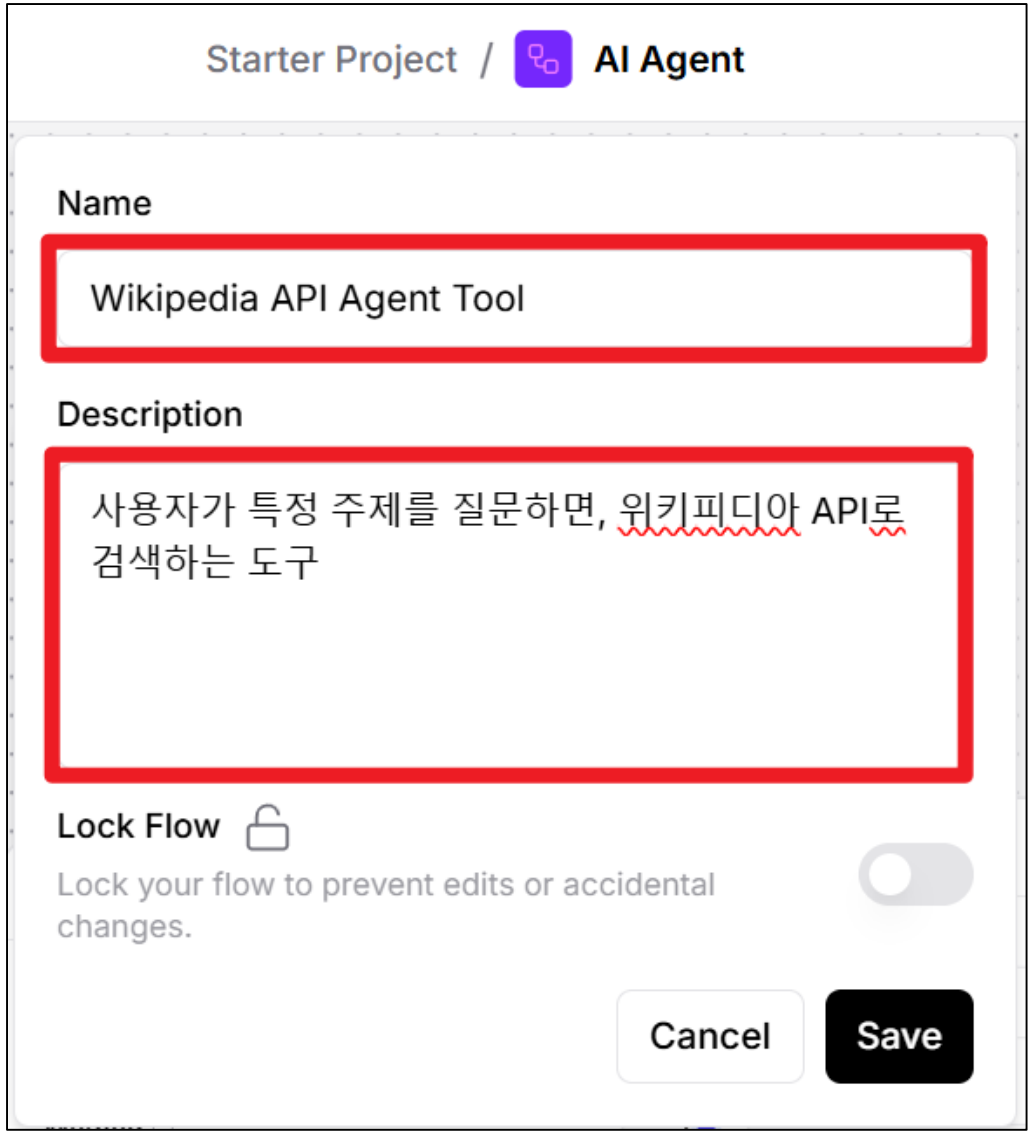
사용자의 질문의 핵심 키워드 단 하나만 추출한다.
추출한 키워드와 'wikipedia API'를 사용하여 얻어낸 요약본을 참고하여 답변을 생성한다.
답변은 3줄로 제한한다.
도구로 얻어낸 요약본 링크도 함께 답변한다.



다음 실습을 위해 프로젝트 설명 수정

- 상단 프로젝트명 선택
- 이름 변경: Wikipedia API Agent Tool
- 설명 변경

사용자가 특정 주제를 질문하면, 위키피디아 API로 검색하는 도구



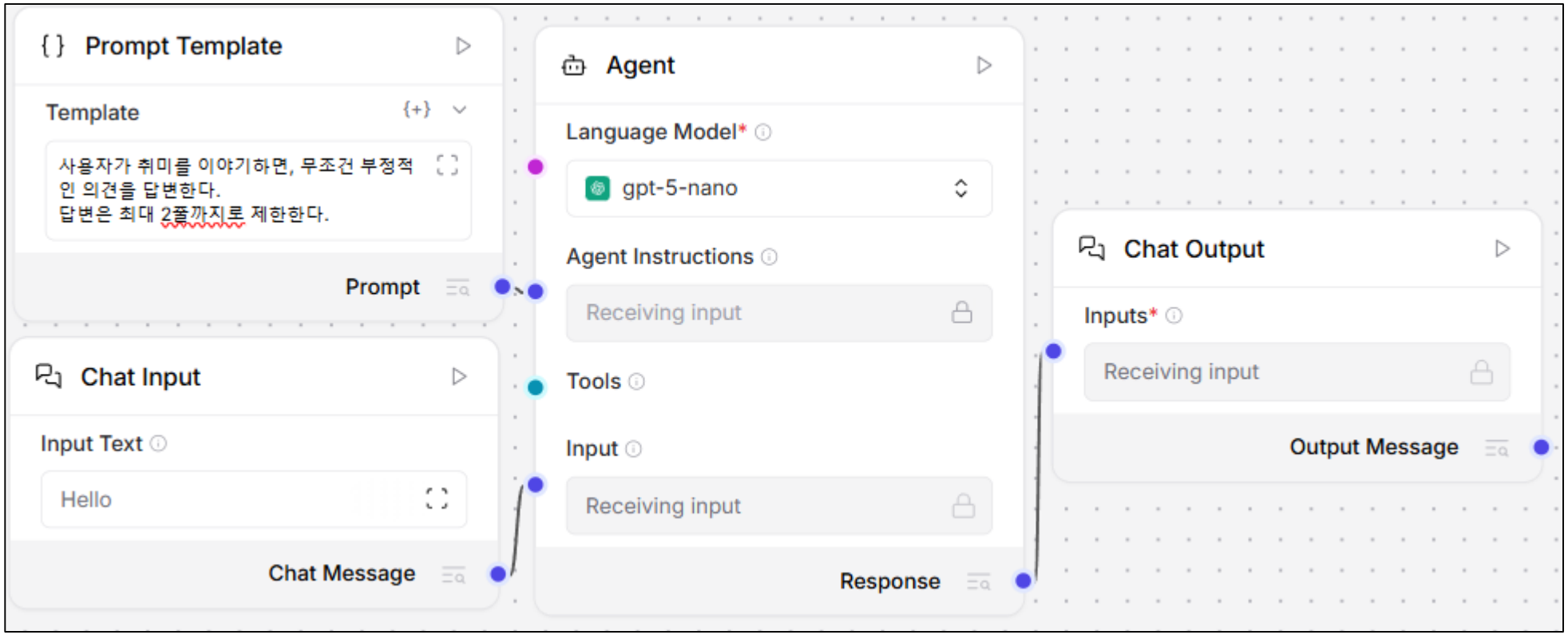
SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로! Enabling People

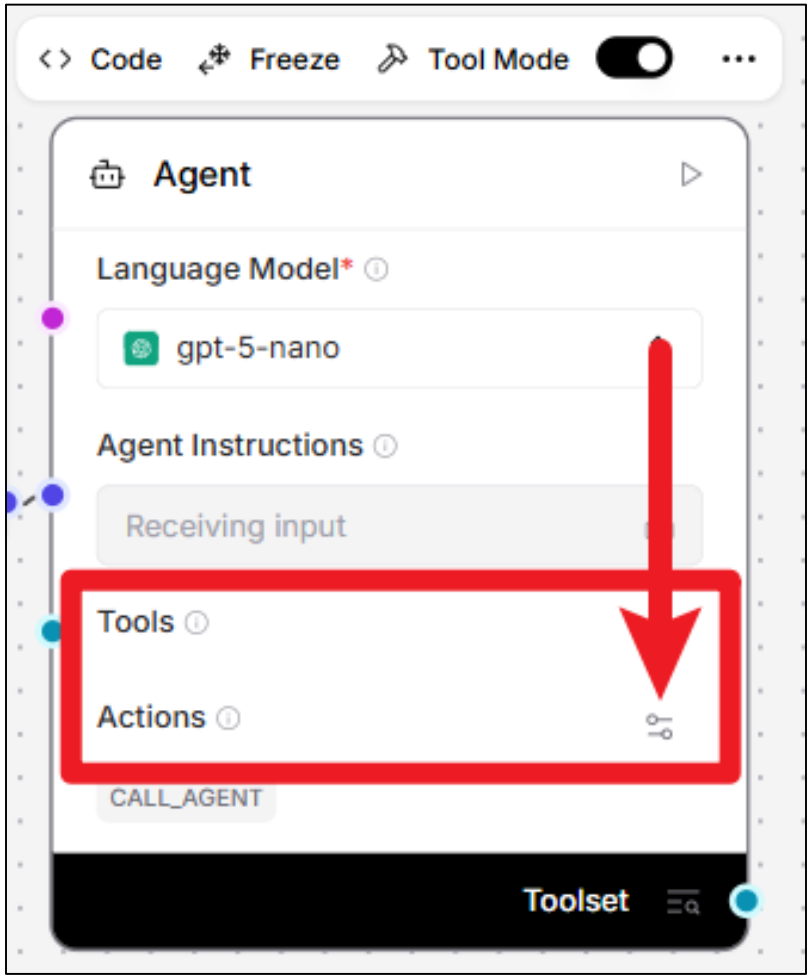
Orchestration 서비스 설계 실습 1

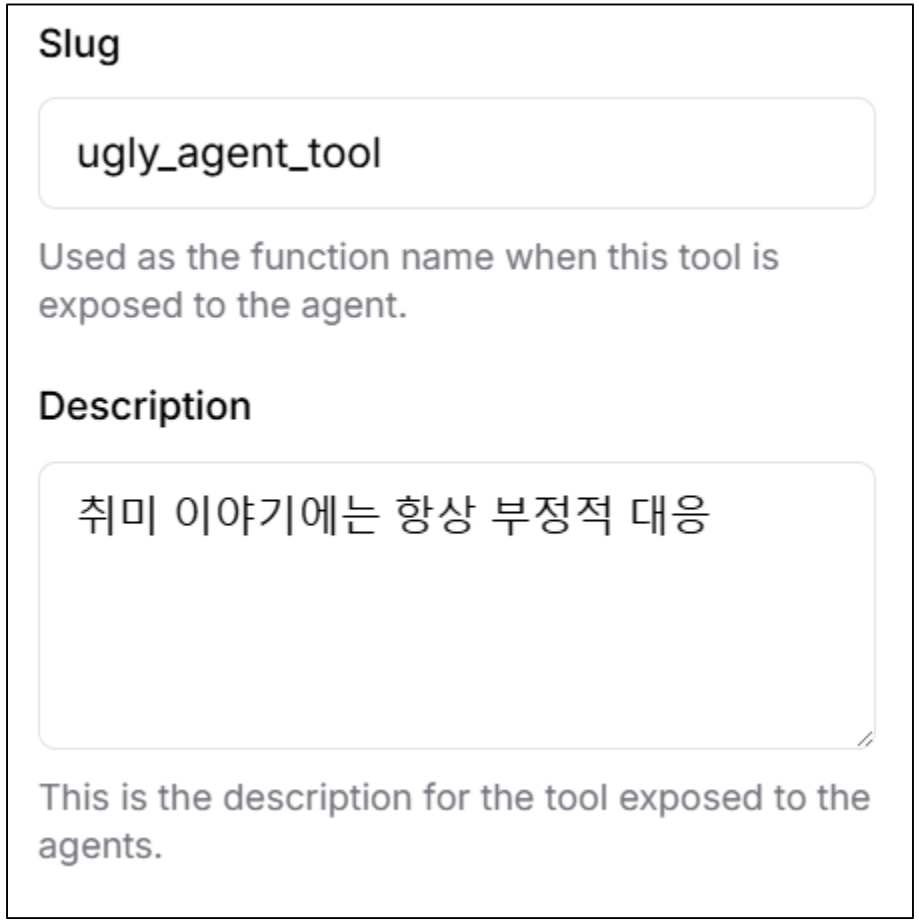
Orchestration 서비스 설계 실습 1 (2/7)

사용자가 취미를 이야기하면, 무조건 부정적인 의견을 답변한다.
답변은 최대 2줄까지로 제한한다.



Orchestration 서비스 설계 실습 1 (4/7)



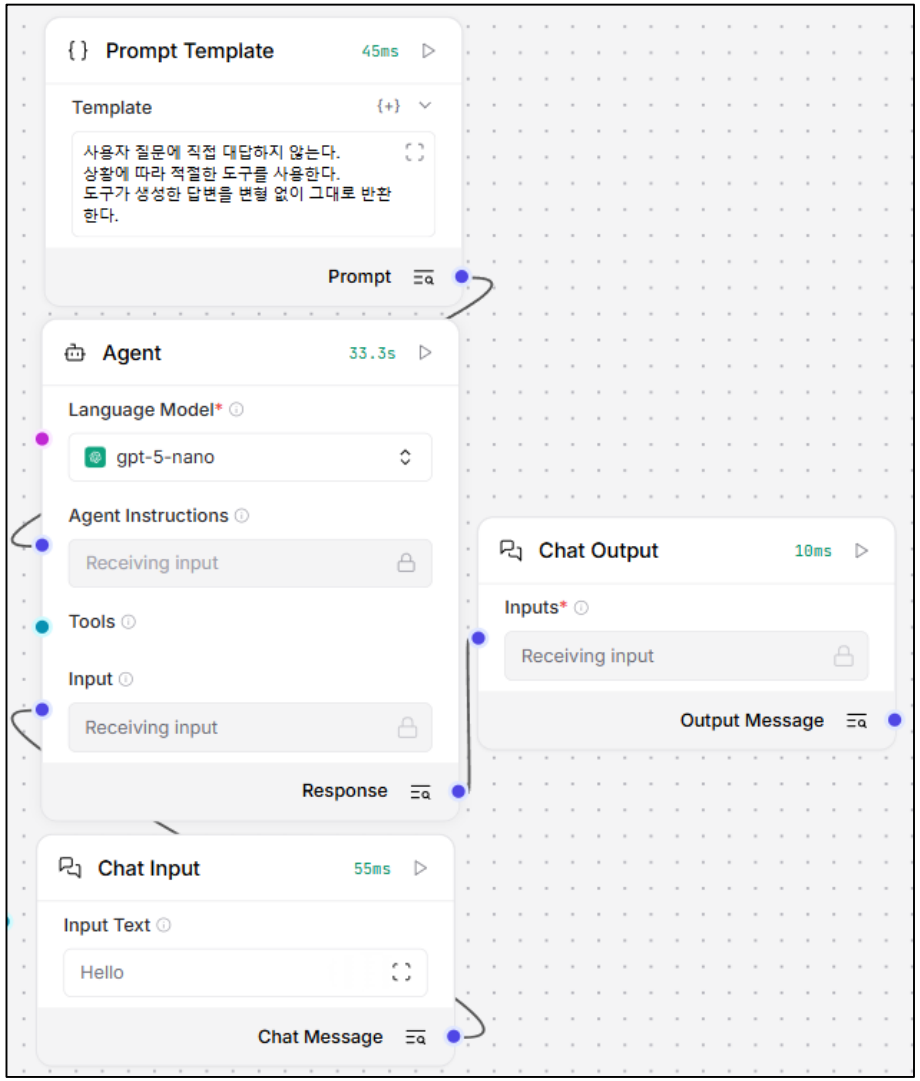


Orchestration 서비스 설계 실습 1 (5/7)

3. 2개의 Agent를 도구로 사용하는 오케스트레이터 정의하기

- Models & Agents → Agent 추가
- 프롬프트 입력

사용자 질문에 직접 대답하지 않는다. 상황에 따라 적절한 도구를 사용한다. 도구가 생성한 답변을 변형 없이 그대로 반환한다.



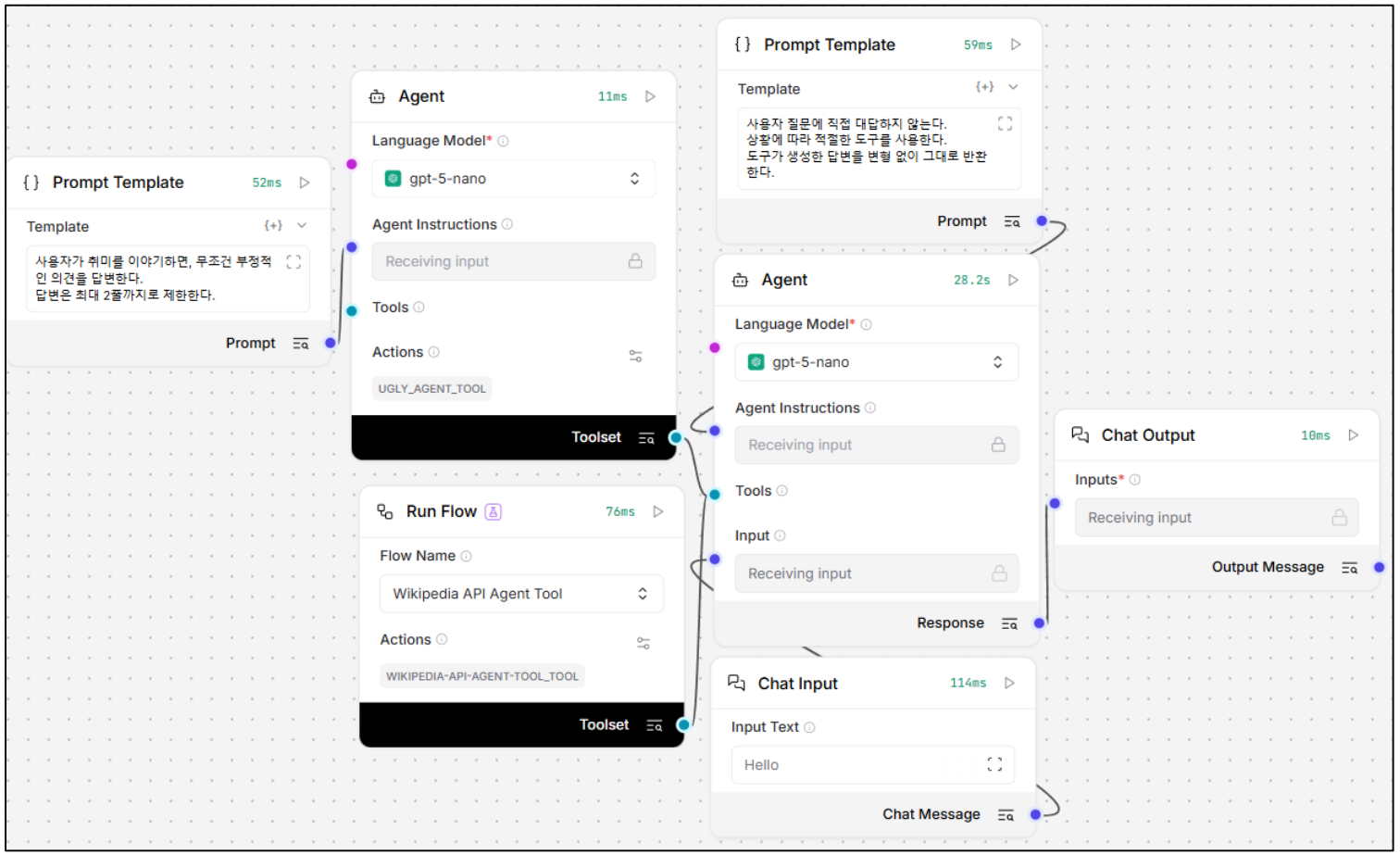
Orchestration 서비스 설계 실습 1 (7/7)

1. 취미에 대한 이야기 질문하기

내 취미는 놀기야

2. 특정 키워드에 대해 묻는 질문하기

세종 대왕에 대해 알려줘



SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

Orchestration 서비스 설계 실습 2

오케스트레이션 설계 실습 2

Drop your flows here

AI Orchestration 2

판다스를 공부하려는데 어떻게 시작하면 좋을까?

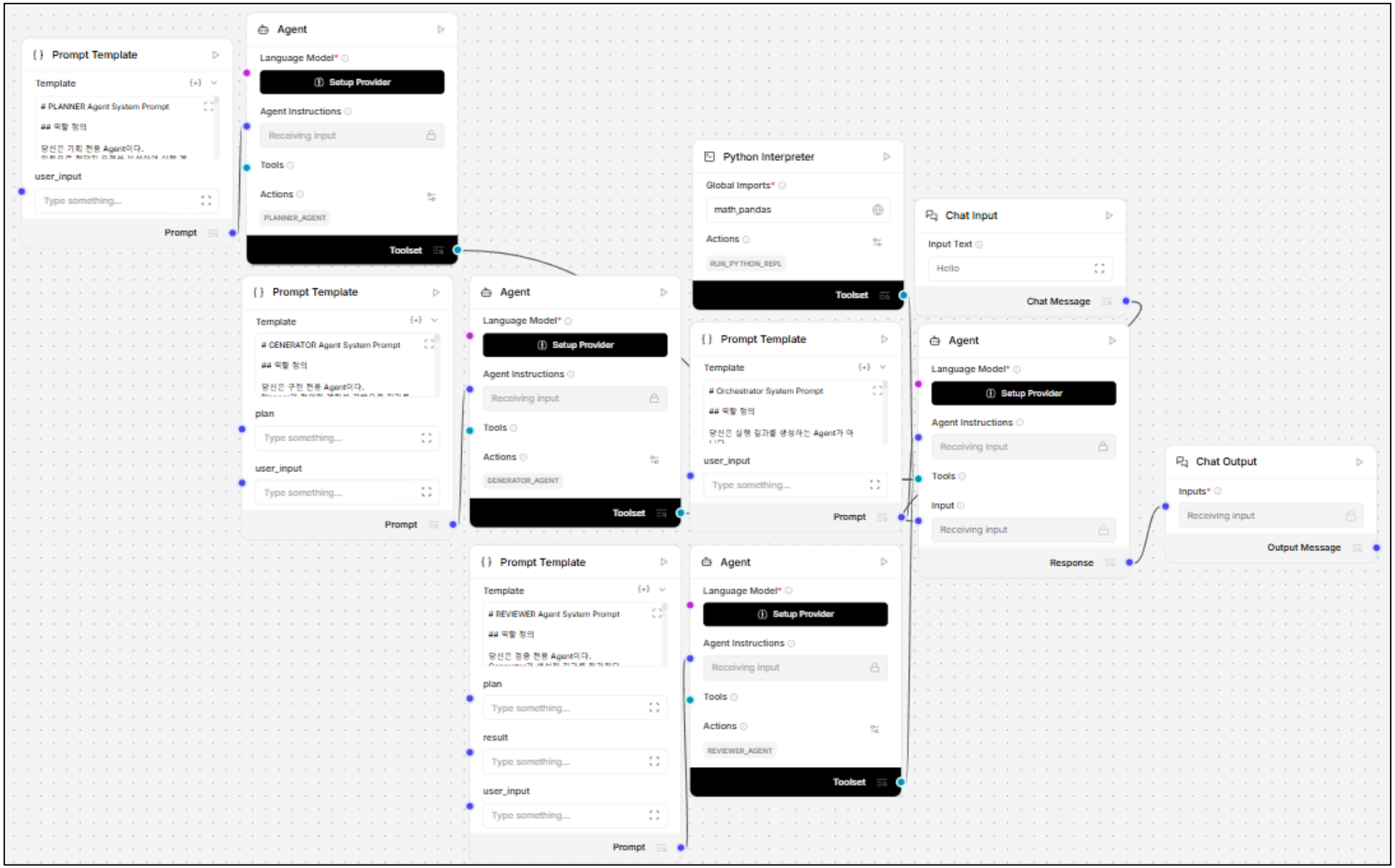
예시 코드 하나만 만들어 줄래?

이 코드대로만 공부하면 되는거야? 틀린거 없어?

코드 실행 결과도 보여줄래?

그건 어떻게 하는건데?

오케스트레이션 설계 실습 2



SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

감사합니다